CS 3210

3.4

1) Let us first convert everything to base 10.

$4365_8 = 4 \times 8^3 + 3 \times 8^2 + 6 \times 8^1 + 5 \times 8^0 = 2293_{10}$
$3412_8 = 3 \times 8^3 + 4 \times 8^2 + 1 \times 8^1 + 2 \times 8^0 = 1802_{10}$

$2293_{10} - 1802_{10} = 491_{10}$

Division	Quotient	Remainder
491/8	61	3
61/8	7	5
7/8	0	7

Hence, we get 753_8 if we read it backwards.

Next, let us verify 753_8 because 5-2=3, 6-1=5 and since 3 is smaller than 4, we carry over 1 from the 4 that's beside 3 so 3 basically becomes $3_8 + 10_8 = 13_8$ and then we convert it to base 10 so 13_8 becomes $1 \times 8^1 + 3 \times 8^0 = 11$ and 11 - 4 = 7 and that's why we get 753_8, which checks it out.

3.5

Over here, each octal digit is 3 bits and the first bit of the octal determines the sign, where 0 represents that it is positive and 1 represents that it is negative. Now, if this bit is from 0 to 3 then we say that the sign bit is 0 (positive), otherwise if it is 1, then we say that is negative, so in our question, we have 4365_8, and over here the first bit is 4, so the signe will be negative and the magnitude of digits is $(4\text{-}4)365_8 = 0365_8$.We also have 3412_8, where the first digit is 3 which means that its sign is positive and can be represented as $+3412_8$. Next, let us try and compute the values.

$(-0365_8) - (+3412_8) = - (0365_8 + 3412_8)$

This gives us 3777_8 with a negative sign overall. Now since 3 represents a negative sign and the overall sign is also negative, then it would make it positive and we don't want that to happen.

Now, since we want to maintain the negative sign, we will add 4 to 3(which can represented as 011) and to make it 7 (which can be represented as 111), since we want to flip the sign bit from 0 to 1 over here and hence the final 12 bit signed magnitude is 7777_8.

3.14

Since we know that the integer has 8 bits so we are going to have 8 iterations. Now, in hardware since each iteration performs 2 operations that are addition and shift multiplicand & multiplier(which are done together), so we can say that each iteration performs 2 steps. Now, since each iteration performs 2 steps, so we can say that 8 iterations will perform 16 steps, and now since each steps has 4 units, we can say that we are going to have 64 units in total since it is the product of 16(Number of steps) and 4 (Number of units).

In software, we perform 3 steps, that are add, shift multiplicand and shift multiplier, and we have 8 iterations again, so we can say that we have 24 steps in total now, and each step has 4 units, so we have 96 time units in total now.

3.17

Let us first convert the operands over here to base 10 integers, which we can do like this:

$0x33 = 3 \times 16^1 + 3 \times 16^0 = 48 + 3 = 51_{10}$
$0x55 = 5 \times 16^1 + 5 \times 16^0 = 80 + 5 = 85_{10}$

Now let us convert it to binary:

$51 = 64 - 8 - 4 - 1 = 2^6 - 2^3 - 2^2 - 2^0$

Now let us multiply 51 x 85, which could be represented as follows:

85 x 51= (85<<6) - (85<<3) -(85<< 2) - 85

This is basically a smarter way as shifting is much faster than multiplying, also this form lets the hardware or compilers reuse the shifts way more efficiently.

Hence our final answer is as follows:

0x33 x 0x55 = (0x55 << 6) - (0x55 << 3) - (0x55 << 2) - 0x55

3.29

Let us each operand to binary 16 first, so we can represent 26.125 as 11010.001_2 in binary. Then if we normalize it then we can represent it as $1.1010001_2 \times 2^4$, then we can see that exponent value over here is 4 where if we store then it becomes e = 4 + 15 = 19 = 10011_2, and in fraction itr represented as 1010001000, so the bits of A are s=0, e=10011 and f = 1010001000.

Similarly we can represent 0.4150390625 in binary as 0.4150390625 = 425/1024 = 0.0110101001_2,if we normalize it then it can be represented as $0.0110101001_2 = 1.10101001_2 \times 2^{-2}$. The exponent value over here is -2, which can be stored in a variable e which contains then value of -2 + 15 =13, which can be represented as 01101_2, which could be represented as 101000100 and hence we can day that B can be represented as s=0, e=01101 and f = 1010100100.

Now let us align the elements first, where we will keep the larger exponent (40 and shift the smaller mantissa right by 6 since 4-(-2) = 4+2 = 6,so represent the mantissas as:

A: 1.1010001000 | G=0,R=0, S=0
B can be shifted right by 6, so we can say that it contributes 0.0000011010100100… and into our 10+3 positions, it maps as bits at positions 6…(6+10…).

Let us now make a 13-bit lane where index positions 1..10 are fraction bits, 11=G, 12=R, 13=S, so we can represent A as 1010001000 | 000 and B as 000011010|100.

Now, let us add the aligned mantissas, so in order to do that we have to add bitwise (from pos13 to pos1), which may result for positions 1..13 as 1010100010|100, so if we unround the mantissa, we get a fraction 1010100010, where G = 1, R = 0 and S = 0, where hidden bit remains 1 → mantissa = 1.1010100010, where exponent stays 4 which means that we don't have any overflow.

Now let us round up to the nearest even using G/R/S, where we keep a 10 fraction bits where the discarded pattern is exactly 100…0, which is a tie case since G =1, R = 0, and S = 0. In the tie rule, we can round up only if the current LSB is 1; however, if the current LSB is 0then we do not bound. So, the final fraction stays 1010100010, which has an exponent of 4.

Hence, the final sign is 0, the exponent is 4+15 =19 = 10011_2, hence the fraction is 1010100010, so the final 16-bit word is 0 10011 1010100010 = 0100 1110 1010 0010_2 = 0x4EA2, and hence the value is $1.1010100010_2 \times 2^4$ = 26.53125.

3.33

Let us first convert all the values to their binary 16 form.

Value	Normalized binary	Exponent	Stored Exp(E+15)	Fraction bits
3.984375 x 10^{-1}=0.3984375	1.10011 x 2^{-2}	-2	13 (01101)	1001100000
3.4375x10^{-1} = 0.34375	1.011 x 2^{-2}	-2	13(01101)	0110000000
1.771 x 10^{3} = 1771	1.1011101011 x 2^{10}	10	25(11001)	1011101011

Now, let us add the inner parenthesis, where the exponent gap is 10 -(-2) = 12, which means that B shifts 12 bits to the right. After shifting, B contributes only towards very small bits (such as guard, round, or sticky), and hence the sum approximately becomes equal to C, since 1771 >> 0.34, which results in no change when rounded off to the nearest even, so our result so far is:

B+C = 1.1011101011_2 x 2^{10}

Now, let us add A. Since we have the same exponent gap, that is 12, A also shifts towards the right now, so adding A doesn't change the stored value after rounding.

Hence, our sign over here is 0, exponent is 25 (11001_2 in binary), fraction is 1011101011_2, which can be represented as 0 11001 1011101011 (0x66EB) in binary 16, so the result is 1771.0_{10}.

3.44

Over here, we know that the BCD uses 4 bits per decimal digit, so we can store 24/4 = 6 digits in the fraction. Now, we know that ⅓ can be written in the following way in its base 10 form:

⅓ = 0.333333…

Since only 6 digits are available, that becomes 0.333333.

Now, each digit 3 in BCD is 0011, but since there are 6 3's we can write it as follows:

0011 0011 0011 0011 0011 0011 0011

No, it is not exact since in base 10, a fraction can only be exact if its reduced denominator has no primes other than 2 or 5. Since, in 1/3 , the denominator is 3 it repeats itself in decimal , so it can't be exact with only 6 digits.

3.45

Over here, we know that a single-base 15 digit must be able to represent values from 0-14, so that fits in 4 bits since it covers bits over 0-15, so 24/4 = 6 base-15 digits. Now, let us write ⅓ in base 15. Let us consider a variable x such that it complies with the following statement:

x/15 = ⅓

x = 5

Hence, we can say that ⅓ = 0.5_{15},so over here we can see that it terminates, so our 6 digits are 0.500000_{15}.

Now, we need to convert each base-15 digit to their 4-bit binary, so we can do it in the following way:

Base-15 digit	Decimal value	4-bit binary
5	5	0101
0	0	0000

Hence, we can say that the 6-digit fraction can be represented as 0101 0000 0000 0000 0000 0000.

Since, ⅓ terminates in base-15, we can say that this is the exact representation.